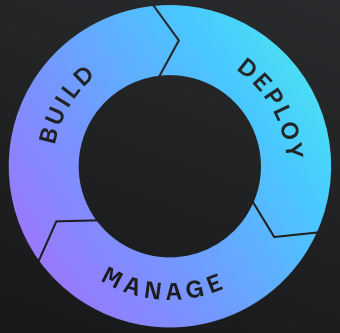
HashiCorp

Infrastructure Lifecycle Management

IaC Accelerator Assessment Guide



Solutions Architect, APJ
Hashicorp

©2024 HASHICORP

Infrastructure Lifecycle Management

IaC Accelerator - maturity assessment guide internal info only

This deck is intended for internal use only

- This deck **IAC-Capability-Maturity-Assessment-Guide** gives guidance on assessment framework
- Each IaC Capability pillar should is to be assessed to benchmark a customers overall IaC capability maturity
- Accompanying deck IAC-Capability-Maturity-Workshop
- Maturity terminology is aligned to devops maturity model / CMM.
- **Note:** This assessment guide is assessing the customers IaC capability.

  The physical architecture topology is not in scope for this, customers should be aligned to HVD architecture.

internal info only

©2024 HASHICORP

IaC Capability Pillars

Automation Workflows

Architect standardized IaC workflows to ensure consistent releases across environments, maximize efficiency and foster collaboration

Quality Standards

Implement and enforce IaC development and testing standards across the organization to ensure consistent quality

Security & Compliance

Establish and enforce robust security measures and compliance guardrails to proactively mitigate risks and ensure policy adherence.

Lifecycle Management

Streamline Day 2 IaC operations to enhance operational efficiency and elevate user experience."

Operational Excellence

Design and implement streamlined onboarding and enablement processes to accelerate IaC adoption and optimize time-to-value

© HASHICORP

Automation Workflows

Assessment mapping

Automation Workflows:

Maturity	IaC Workflow	Collaboration
1 - Ad Hoc	No defined IaC workflow. Manual infrastructure changes. Infrastructure changes initiated by user using developer driven plan and apply	No team alignment for IaC. No shared module library. Lack of RBAC consideration. Local state management
2 - Developing	Basic Git structure defined. Inconsistent branching strategy, duplication of code with unclear release process across environments. Releases may require manual code intervention or copy changes due to unstructure release workflow.	Some team structure around IaC. Ad-hoc module sharing. Basic RBAC.
3 - Defined	Documented IaC workflow. Basic branching strategy. Using common code with environment specific variables.	Defined team roles for IaC. Shared module library initiated. RBAC implemented.
4 - Managed	Automated GitOps driven IaC. Enforced branch protection. Consistently structured repos and consistent release process across environments	IaC-aligned team structure. Managed shared modules. RBAC considers self-service consumption with guard rails
5 - Optimized	Standardized GitOps driven IaC. Git structure is provided via self-service using IaC and integrated with secrets management without additional consumer integration effort Branch protect rules are enforce and git approval required by policy for critical workloads	Cross-functional IaC teams. Shared optimized module library - managed as a product with continual improvement and collaborative feedback. RBAC considers self-service consumption with guard rails. Module consumption enforced through policy controls.

©2024 HASHICORP

Quality Standards

Assessment mapping

Quality standards capability:

Maturity	Testing	Publishing	Versioning	Standards	Templates
1 - Ad Hoc	No testing framework in place; testing is manual and developer initiated through via plan and apply. Tests not written for modules.	Not publishing modules to a private registry, sourced from git or use or local modules using latest version	No versioning strategy, left to individual developer	No established coding standards or guidelines.	No templates exist for module or policy development; all configurations are ad hoc.
2 - Developing	Inconsistent application of testing, using terraform test in parts.	Manual module publishing process to a private registry, inconsistent publishing standards (example usage of docs and examples), not all consumers using version constraints	Initial versioning attempts are inconsistent; some teams use semantic versioning but with no enforcement. Consumers not provided clear guidelines on use of constraints	Some initial coding standards introduced, but adoption and enforcement are inconsistent.	Initial templates created for modules, but usage is inconsistent, and they lack any integration.
3 - Defined	Consistent Test approach defined but not fully implemented	Documented publishing process, some attempts at automation but not fully integrated.	Use of versioning is defined and follows semantic versioning. Use of version constraints by consumers inconsistent	Coding standards are documented and beginning to be enforced; not all teams are aligned.	Templates for modules are defined and used, but integration into development workflows is incomplete; teams use them inconsistently.
4 - Managed	Documented Test Framework, Testing is leveraged during module development process. Not yet integrated into module publishing to gate release quality.	Publishing process is mostly automated and integrated with CI/CD pipelines; manual overrides minimal, but testing not enforced in publishing.	Consistent use of semantic versioning, with standards in place for module producers and consumers. Consumer module lifecycle and deprecation not tracked over time.	Documented internal code style guide. Consistently applied.	Standardized templates are used across teams for modules and policies; partially integrated into publishing processes.
5 - Optimized	Automated testing integrated to publishing process, negative testing for input validation; test process also encompasses - consumer example tests with policy, module testing includes provider constraint ranges.	Automated and efficient publishing process, testing enforced as part of publishing process through CICD, ensuring consistent and high quality releases. Publishing reviews enforced through branch protection and approvals.	Consistent use of semantic versioning, with standards in place for module producers and consumers. Module lifecycle and deprecation monitored and tracked with active comms to consumers for uplift	Documented internal code style guide, best practices are continuously refined, ensuring rigorous adherence across teams. Automated documentation to standard. Consumer module consumption enforced through policy	Git templates covering consumer, module and policy repositories. templates are consumed self-service and integrated with secrets management (Vault) using dynamic secrets for tokens

©2024 HASHICORP

Security and Compliance

Assessment mapping

Security and Compliance:

Maturity	Cloud credentials	Pipeline secrets	Secret scanning	Policy controls	Cost controls
1 - Ad Hoc	Lack of standardised approach to cloud credentials. Use of secrets in sensitive variables. Distributed secrets management left to consumers with lack of central control.	Decentralised static credentials in pipeline secrets, lack of centralised control	No controls or standards	No policy controls in place	No cost controls in place
2 - Developing	Using static secrets in environment variables. Credential inherited through machine identity for runners of agent, but traversal risks due to shared runners and identity.	Secrets are centrally managed in secrets management tool. Though consumption is not standardised or enforced. Each team is self-governed as to how they utilised secrets within their pipelines	General guidelines documented enforced through manual code review only	Basic IaC policies exist but are not consistently enforced; manual reviews are common	Basic cost monitoring, but not integrated into IaC workflow
3 - Defined	Centrally controlled secrets, Use of static secrets in environment variables, though rotation not considered or irregular	Pipeline credentials are centrally managed in secrets management and synchronized to pipelines, though rotation not considered	General guidelines documented some automated scanning tools in place, alert based only	Policies are documented and enforced through manual processes	Cost reporting and basic budgeting defined but yet to integrate into IaC workflow
4 - Managed	Cloud credentials are managed through secret management tools though authentication approach relies on environment variables, additionally some credentials are stored in state through use of datasources.	Pipeline credentials are centrally managed in secrets management and synchronized to pipelines and rotated on regular interval	General guidelines documented some automated scanning tools in place across the git organization. Incident based integration, remediation	Policies are enforced automatically using tools. Hard policy requirements are enforced.	Cost controls defined and integrated into IaC workflow, with automated budget alerts
5 - Optimized	Cloud credentials are managed through centralised secret management tool. Authentication to secrets managed tools via trusted identity auth flow. Credential are just-in-time dynamic credentials, no credentials stored in state.	Pipeline secrets centrally managed in secrets management , authenticating via trusted pipeline identity with no stored credentials	Proactive secret scanning, preventing secrets being committed to code. Security incident integration and automatic remediation where possible	Fully automated policy enforcement with continuous monitoring and improvement. Modules factor in organization policy where there are hard requirements. IaC policy controls consider cloud but also cloud agnostic consumption controls	Advanced cost optimization strategies, including predictive analysis and automated optimization, fully integrated into IaC workflow

©2024 HASHICORP

Lifecycle Management

Assessment mapping

Lifecycle Management:

Maturity	IaC lifecycle	Module lifecycle	Platform lifecycle (TFE only)
1 - Ad Hoc	No formal process for managing IaC drift	No tracking of module consumption.	No formal process for platform upgrades.
2 - Developing	Manual processes to detect drift; corrections are reactive	Basic tracking of module usage; no formal deprecation process.	Manual upgrades with some documentation; inconsistent practices.
3 - Defined	Documented processes for drift management; some automation in place.	Defined process for module lifecycle management, including deprecation policies.	Documented upgrade process; manual but consistent execution.
4 - Managed	Automated detection and correction of drift; integrated into workflow	Automated tracking and management of module lifecycle; regular audits.	Automated upgrade processes; minimal downtime. Irregular cadence
5 - Optimized	Continuous improvement; proactive drift management; fully integrated alerting and notifications	Advanced analytics for module usage; proactive lifecycle management. Defined policies around module lifecycle, standardised communication channels to manage module deprecation and consumer notifications	Automated upgrade processes; minimal downtime; Regular cadence (usually adoption new version within 3 months)

©2024 HASHICORP

Operational Excellence

Assessment mapping

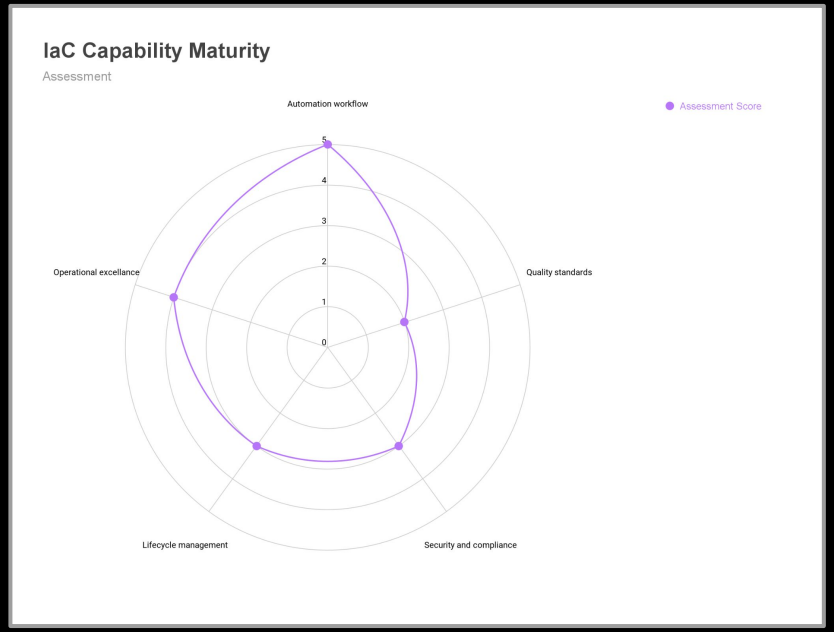
Operational Excellence:

Maturity	Tenant Onboarding	Tenant Enablement and Communications
1 - Ad Hoc	Manual onboarding process. No automation. No self-service options. Request based then manually configured	No formal induction or training. Limited or no documentation available.
2 - Developing	Partially automated onboarding process. Some manual steps remain.	Basic induction provided. Limited documentation and training resources.
3 - Defined	Fully documented onboarding process. Some automation, but not fully self-service.	Structured introduction process. Comprehensive documentation available.
4 - Managed	Mostly automated onboarding process. Self-service options available.	Comprehensive training resources. Regular cadence with consumer to capture feedback and prioritise features and bugfixes.
5 - Optimized	Fully automated, self-service onboarding process. IaC-driven. Single integrated onboarding process where possible across cloud, IaC and Secrets management (assuming greenfields consumer)	Comprehensive training resources. Regular cadence with consumers to capture feedback and prioritise features and bugfixes. Community and inner sourcing model established to share thought leadership within the organisation. IaC platform managed as a product with dedicated product owner.

©2024 HASHICORP

Example IaC Maturity

Example Assessment chart



The assessment guide is used as a means of benchmarking a customers IaC capability, with a view to prioritize uplift of capability and increase customer adoption.

[IaC Foundation Pillars - Example chart]

IaC Capability Pillars	Assessment Score (1-5)
Automation workflow	5
Quality standards	2
Security and compliance	3
Lifecycle management	3
Operational excellence	4

©2024 HASHICORP

Overview

Challenges Identified

Item1:

Recommendation:

Item2:

Recommendation:

Item3:

Recommendation:

©2024 HASHICORP

Example IaC Maturity

Recommendations Summary - WIP

IaC Capability Pillars	Assessment (1-5)	Recommendations	Priority	Action Item - Next Steps
Automation workflow	5	none - optimized		
Quality standards	2	Recommendation 1 - Testing/module publishing - Establish standard test framework and integrated into an automated module publishing workflow	**High - 2 - improve module quality**	
Security and compliance	3	Recommendation 2 - Security - Adoption of dynamic provider credentials for AWS	**High - 1 - reason improve security posture**	
Lifecycle management	3	Recommendation 3 - platform lifecycle cadence	**Medium - 3 -**	
Operational excellence	4			

©2024 HASHICORP

Example IaC Maturity

Identified Risks - WIP

IaC Capability Pillars	Risks	Priority	Action Item - Next Steps
Automation workflow			
Quality standards		High - 1	
Security and compliance		High - 2	
Lifecycle management		Medium - 3	
Operational excellence			

©2024 HASHICORP

Example IaC Maturity

Next Steps

- Workshop A - Item
- Capability Uplift - Resident Solution Architect - embedded capability

©2024 HASHICORP

Thank you

hello@hashicorp.com

©2024 HASHICORP